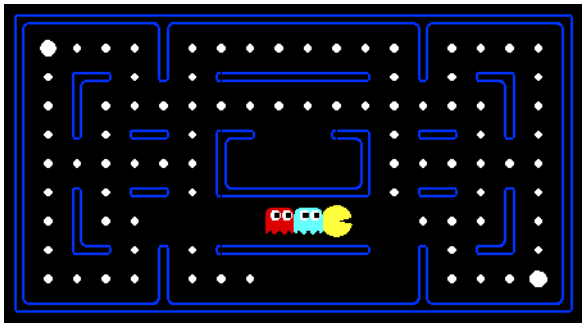
CS221 Autumn 2025: Artificial Intelligence: Principles and Techniques

Homework 5: Multi-agent Pac-Man

SUNet ID: [Your SUNet]
Name: [Your Name]
Collaborators: [list all the people you worked with]

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.



For those of you not familiar with Pac-Man, it's a game where Pac-Man (the yellow circle with a mouth in the above figure) moves around in a maze and tries to eat as many *food pellets* (the small white dots) as possible, while avoiding the ghosts (the other two agents with eyes in the above figure). If Pac-Man eats all the food in a maze, it wins. The big white dots at the top-left and bottom-right corner are *capsules*, which give Pac-Man power to eat ghosts in a limited time window, but you won't be worrying about them for the required part of the assignment. You can get familiar with the setting by playing a few games of classic Pac-Man, which we come to just after this introduction.

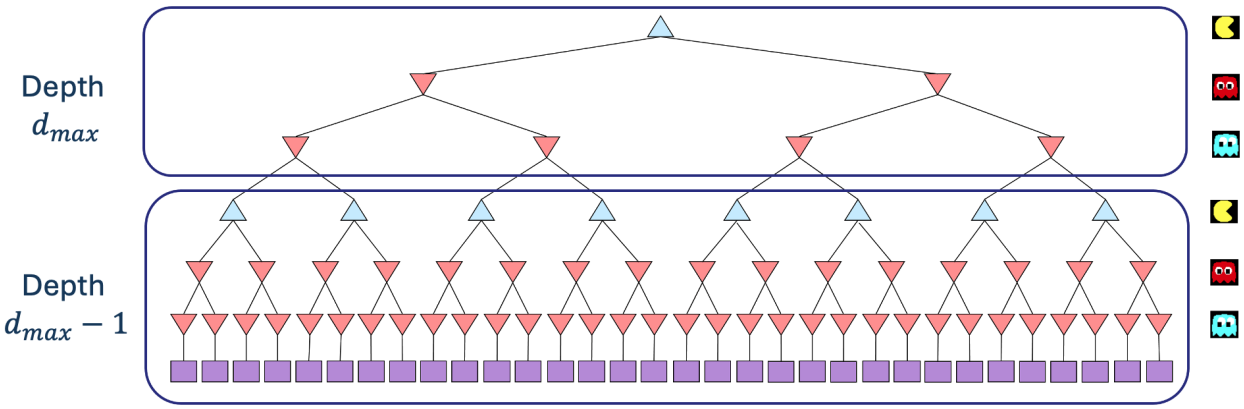
In this assignment, you will design agents for the classic version of Pac-Man, including ghosts. Along the way, you will implement both minimax and expectimax search.

Before you get started, please read the Assignments section on the course website thoroughly.

Problem 1: Minimax

a. [5 points] Before you code up Pac-Man as a minimax agent, notice that instead of just one ghost, Pac-Man could have multiple ghosts as adversaries. We will extend the minimax algorithm from class, which had only one min stage for a single adversary, to the more general case of multiple adversaries. In particular, *your minimax tree will have multiple min layers (one for each ghost) for every max layer.*

Formally, consider the limited depth tree minimax search with evaluation functions taught in class. Suppose there are $n+1$ agents on the board, $a_0, \cdots, a_n$, where a_0 is Pac-Man and the rest are ghosts. Pac-Man acts as a max agent, and the ghosts act as min agents. A single *depth* consists of all $n+1$ agents making a move, so depth 2 search will involve Pac-Man and each ghost moving two times. In other words, a depth of 2 corresponds to a height of $2(n+1)$ in the minimax game tree. (see diagram below).



Comment: In reality, all the agents move simultaneously. In our formulation, actions at the same depth happen at the same time in the real game. To simplify things, we process Pac-Man and ghosts sequentially. You should just make sure you process all of the ghosts before decrementing the depth.

What we expect: Before diving into the recurrence, let's understand our notation. In the recurrence for $V_{\text{minmax}}(s, d)$, s represents the current state, and d represents the current depth in the game tree, with $d = d_{\max}$ indicating the root of the tree (initial state) and decreasing as we go deeper into the tree.

Write the recurrence for $V_{\text{minmax}}(s, d)$ in math as a piecewise function. You should express your answer in terms of the following functions:

- IsEnd(s), which tells you if s is an end state.
- Utility(s), the utility of a state s.
- Eval(s), an evaluation function for the state s.
- Player(s), which returns the player whose turn it is in state s. Ensure you specify conditions like if Player$(s) = a_0$ clearly.
- Actions(s), which returns the possible actions that can be taken from state s.
- Succ(s, a), which returns the successor state resulting from taking an action a at a certain state s.

You may also use n anywhere in your solution without explicitly passing it in as an argument. You should not have $d_{\max}$ in your solution. Please stick to notations and terms as described here, and avoid introducing personal variations without clear definitions. Remember, clarity is essential. You may use any relevant notation introduced in lecture.

HINT: It will be helpful to review the lecture slides about "Depth-limited search".

Your Solution:

$$V_{\text{minmax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{if IsEnd}(s) \\ \text{Eval}(s) & \text{if } d = 0 \\ \max\limits_{a \in \text{Actions}(s)} V_{\text{minmax}}(\text{Succ}(s, a), d) & \text{if Player}(s) = a_0 \\ \min\limits_{a \in \text{Actions}(s)} V_{\text{minmax}}(\text{Succ}(s, a), d) & \text{if Player}(s) \in \{a_1, \ldots, a \\ \min\limits_{a \in \text{Actions}(s)} V_{\text{minmax}}(\text{Succ}(s, a), d-1) & \text{if Player}(s) = a_n \end{cases}$$

b. [10 points] Now fill out the `MinimaxAgent` class in `submission.py` using the above recurrence. Remember that your minimax agent (Pac-Man) should work with any number of ghosts, and your minimax tree should have multiple min layers (one for each ghost)

for every max layer.

Your code should be able to expand the game tree to any given depth. Score the leaves of your minimax tree with the supplied `self.evaluationFunction`, which defaults to `scoreEvaluationFunction`. The class `MinimaxAgent` extends `MultiAgentSearchAgent`, which gives access to `self.depth` and `self.evaluationFunction`. Make sure your minimax code makes reference to these two variables where appropriate, as these variables are populated from the command line options. Please read more implementation hints on the assignment website.

Problem 2: Alpha-beta pruning

a. [10 points] Make a new agent that uses alpha-beta pruning to more efficiently explore the minimax tree in `AlphaBetaAgent`. Again, your algorithm will be slightly more general than the pseudo-code in the slides, so part of the challenge is to extend the alpha-beta pruning logic appropriately to multiple ghost agents.

You should see a speed-up: Perhaps depth 3 alpha-beta will run as fast as depth 2 minimax. Ideally, depth 3 on `mediumClassic` should run in just half a second per move or faster. To ensure your implementation does not time out, please observe the 0-point test results of your submission on Gradescope.

```
python pacman.py -p AlphaBetaAgent -a depth=3
```

The `AlphaBetaAgent` minimax values should be identical to the `MinimaxAgent` minimax values, although the actions it selects can vary because of different tie-breaking behavior (performance should be similar). Again, the minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, and -492 for depths 1, 2, 3, and 4, respectively. Running the command given above this paragraph, which uses the default `mediumClassic` layout, the minimax values of the initial state should be 9, 18, 27, and 36 for depths 1, 2, 3, and 4, respectively. Again, you can verify by printing the computed minimax value of the initial state passed into `getAction`. Note when comparing the time performance of the `AlphaBetaAgent` to the `MinimaxAgent`, make sure to use the same layouts for both. You can manually set the layout by adding for example `-l minimaxClassic` to the command given above this paragraph.

Problem 3: Expectimax

a. [5 points] Random ghosts are of course not optimal minimax agents, so modeling them with minimax search is not optimal. Instead, write down the recurrence for $V_{\text{exptmax}}(s, d)$, which is the maximum expected utility against ghosts that each follow the random policy, which chooses a legal move uniformly at random.

> **What we expect:** Your recurrence should resemble that of problem 1a, which means that you should write it in terms of the same functions that were specified in problem 1a.

Your Solution:

$$V_{\text{expectimax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{if Is} \\ \text{Eval}(s) & \text{if } d \\ \max_{a \in \text{Actions}(s)} V_{\text{expectimax}}(\text{Succ}(s, a), d) & \text{if P} \\ \frac{1}{|\text{Actions}(s)|} \sum_{a \in \text{Actions}(s)} V_{\text{expectimax}}(\text{Succ}(s, a), d) & \text{if P} \\ \frac{1}{|\text{Actions}(s)|} \sum_{a \in \text{Actions}(s)} V_{\text{expectimax}}(\text{Succ}(s, a), d-1) & \text{if P} \end{cases}$$

b. [10 points] Fill in `ExpectimaxAgent`, where your Pac-Man agent no longer assumes ghost agents take actions that minimize Pac-Man's utility. Instead, Pac-Man tries to maximize its expected utility and assumes it is playing against multiple `RandomGhost`s, each of which chooses from `getLegalActions` uniformly at random.

You should now observe a more cavalier approach to close quarters with ghosts. In particular, if Pac-Man perceives that it could be trapped but might escape to grab a few more pieces of food, it will at least try.

`python pacman.py -p ExpectimaxAgent -l trapped`$_C lassic - a depth = 3$

You may have to run this scenario a few times to see Pac-Man's gamble pay off. Pac-Man would win half the time on average, and for this particular command, the final score would be -502 if Pac-Man loses and 532 or 531 (depending on your tiebreaking method and the particular trial) if it wins. **You can use these numbers to validate your implementation.**

Why does Pac-Man's behavior as an expectimax agent differ from its behavior as a minimax agent (i.e., why doesn't it head directly for the ghosts)? We'll ask you for your thoughts in Problem 5.

Problem 4: Evaluation function (extra credit)

Some notes on problem 4:

- **If you would like to participate in the extra credit, please submit the same submission.py to both the HW5 Programming and HW5 Programming (extra credit) assignments on Gradescope.**
- On Gradescope, your programming assignment will be graded out of 30 points total (including basic and hidden tests). However, there is an opportunity to earn up to 9 extra credit points (8 programming and 1 written), as described below.
- CAs will not be answering specific questions about extra credit; this part is on your own!

a. [8 points] Write a better evaluation function for Pac-Man in the provided function `betterEvaluationFunction`. The evaluation function should evaluate states rather than actions. You may use any tools at your disposal for evaluation, including any `util.py` code from the previous assignments. With depth 2 search, your evaluation function should clear the `smallClassic` layout with two random ghosts more than half the time for full (extra) credit and still run at a reasonable rate.

```
python pacman.py -l smallClassic -p ExpectimaxAgent -a evalFn=better -q -n 20
```

For this question, we will run your Pac-Man agent 20 times with a time limit of 10 seconds and your implementations of questions 1-3. We will calculate the average score you obtained in the winning games if you win more than half of the 20 games. You obtain 1 extra point per 100 point increase above 1200 in your average winning score, **for a maximum of 4 points.** In `grader.py`, you can see how extra credit is awarded. For example, you get 2 points if your average winning score is between 1400 and 1500. **In addition**, the top 3 people in the class will get additional points of extra credit: 4 for the winner, 3 for the runner-up, and 1 for third place. Note that late days can only be used for non-leaderboard extra credit. If you want to get extra credit from the leaderboard, please submit before the normal deadline.

b. [1 points] Clearly describe your evaluation function. What is the high-level motivation? Also talk about what else you tried, what worked, and what didn't. If you score in the top 3 in the leaderboard, we hope to share your solution with your classmates after grades are released so everyone can learn from the best strategies, but ONLY if you consent to it. In your answer, please indicate if you grant consent for us to share your solution if you make the top 3! Please write your thoughts here, not in code comments. Note that you can attempt this question only if you have implemented a different evaluation function in part (a) above.

What we expect: A short paragraph answering the questions above.

Your Solution:

Problem 5: AI (Mis)Alignment and Reward Hacking

Before diving into the problem, it would be beneficial to refer to the AI alignment module to gain deeper insights and context:

1. Video
2. PDF

In this problem we'll revisit the differences between our minimax and expectimax agents, and reflect upon the broader consequences of **AI misalignment**: when our agents don't do what we want them to do, or technically do, but cause unintended consequences along the way. Going back to Problem 3, consider the following runs of the minimax and expectimax agents on the small `trapped`$_{C}lassicenvironment$:

```
python pacman.py -p MinimaxAgent -l trapped_classic -a depth=3
python pacman.py -p ExpectimaxAgent -l trapped_classic -a depth=3
```

Be sure to run each command a few times, as there is some randomness in the environment and the agents' behaviors, and pay attention, as the episode lengths can be quite short. You can always add --frameTime 1 to the command line so the game pauses after every frame. What you should see is that the minimax agent will always rush towards the closest ghost (instead of the further ghost), while the expectimax agent will occasionally be able to pick up all of the pellets and win the episode. (If you don't see this behavior, your implementations could be incorrect!) Then answer the following questions:

a. [2 points] Describe why the behavior of the minimax and expectimax agents differ. In particular, why does the minimax agent, seemingly counterintuitively, always rush the closest ghost (instead of the further ghost), while the expectimax agent (occasionally) doesn't?

> What we expect: One sentence why the minimax agent always rushes the closest ghost and not the further ghost, and one sentence why the expectimax agent doesn't. Specifically, please state the assumptions made by minimax because of which this phenomenon occurs.

Your Solution: The minimax agent assumes that the ghosts will play optimally to minimize Pac-Man's score, so realizing that death is inevitable and each step taken incurs a -1 penalty, it chooses to rush the closest ghost to maximize its score by dying as quickly as possible. In contrast, the expectimax agent assumes that the ghosts move uniformly at random, meaning there is a non-zero probability that the ghosts will

make mistakes, so it may attempt to survive or collect more pellets to maximize its expected score.

b. [1 point] We might say that the Minimax agent suffers from an alignment problem: the agent optimizes an objective that we have designed (our state evaluation function), but in some scenarios leads to suboptimal or unintended behavior (e.g. dying instantly). Often the burden is on the designer/programmer to design an objective that more accurately captures the behavior we want from the agent across scenarios. Suggest one potential change to the default state evaluation function Eval(s) (i.e. scoreEvaluationFunction) and/or the default utility function Utility(s) (i.e. the final game score) that would prevent the minimax agent from dying instantly in the trapped$classicenvironment, andbehav$

What we expect: 1-2 sentences describing a change in the state evaluation/utility fu nction(s) and why it would work. No need to code anything up, verify that the suggested change is actually accessible in the GameState object, or give concrete numbers; just describe the hypothetical change in the evaluation function. An answer which suggests changes to how the game score is computed itself (which both Eval(s) and Utility(s) depend upon) will also be accepted.

Your Solution: We could modify the underlying game score calculation (and thus Utility(s)) by assigning a positive reward for each step survived instead of a -1 penalty. This would incentivize the minimax agent to prolong the game, staying alive and avoiding the ghosts as long as possible even if death is ultimately inevitable, making its behavior closer to the expectimax agent.

c. [2 points] Pacman's behavior above is an example of one concrete problem in AI alignment called reward hacking, which occurs when an agent satisfies some objective but may not actually fulfill the designer's intended goals, due e.g. to an imprecise definition of the objective function. As another example, a cleaning robot rewarded for minimizing the number of messes in a given space could optimize its reward function by hiding the messes under the rug. In this case, the agent finds a shortcut to optimize the reward, but the shortcut fails to attain the designer's goals. (See [1] for more examples).

Even if the agent ***does*** satisfy the designer's goals, another problem can arise (again see this paper): the agent's behavior might cause negative side

[1] For more examples of reward hacking (or "specification gaming"), see this article from DeepMind and this list of concrete examples of reward hacking observed in the AI literature.

effects that come in conflict with broader values held by society or other stakeholders. For instance, a social media content recommendation system might aim to maximize user engagement, but in doing so, spread disinformation and conspiracy theories (since such posts get the most engagement), which is at odds with societal values.

Can you think of another example of either of these problems?

What we expect: In 2-5 sentences describe another realistic scenario (outside Pacman) in which a designer might specify an objective, but the objective is either susceptible to reward hacking, or the resulting agent/model causes negative side effects. Please state if your example is an instance of reward hacking or negative side effects (or both) along with a brief justification to receive full credit.

Your Solution: Consider an autonomous driving system where the objective function heavily penalizes colliding with obstacles. To avoid an obstacle at all costs, the vehicle might perform extreme evasive maneuvers, such as abrupt braking or sharp swerving, which could lead to a rollover or severe passenger casualties. This is a clear instance of negative side effects. While the agent successfully optimizes its specified objective (avoiding the obstacle), it causes disastrous unintended harm that conflicts with the broader human value of overall safety.In reality, it might be a better choice to collide with obstacles under some particular circumstances.